Continuous-Time Multi-State Actuarial Valuation Using Gompertz-Calibrated Mortality and Age-Dependent Markov Models

Do Binh Kiet

April 21st, 2026

Introduction

This project develops a continuous-time actuarial framework for valuing insurance liabilities using tools from Linear Algebra and stochastic processes. The model combines:

- Gompertz mortality calibration from real data,
- Multi-state continuous-time Markov chains,
- Matrix exponentials for transition probabilities,
- Numerical valuation of expected present value (EPV).

Mortality data is obtained from the **UN World Population Prospects (2024 Revision)**, specifically the 2023 abridged life tables for Vietnam.

Mortality Modeling

Mortality intensity is modeled using the Gompertz law:

$$\mu(x) = Ae^{Bx}$$

Taking logarithms:

$$\log \mu(x) = \log A + Bx$$

This allows estimation of parameters via linear regression.

Estimated Parameters

Population	A	B
Female	0.00003875	0.091607
Male	0.00015140	0.081907
Both	0.00010188	0.082458

Mortality doubling time is given by:

$$\frac{\ln 2}{B}$$

Multi-State Model

We consider a three-state system:

- Healthy (1)
- Disabled (2)
- Dead (3, absorbing)

Generator Matrix

$$Q(x) = \begin{bmatrix} -(\lambda_{12} + \mu(x)) & \lambda_{12} & \mu(x) \\ \lambda_{21} & -(\lambda_{21} + \mu_d(x)) & \mu_d(x) \\ 0 & 0 & 0 \end{bmatrix}$$

Parameter Assumptions

The disability transition parameters are not calibrated from Vietnamese data, but assumed from actuarial literature:

$$\lambda_{12} = 0.02, \quad \lambda_{21} = 0.10, \quad \mu_d(x) = 2\mu(x)$$

Linear Algebra Framework

Transition probabilities are computed using the matrix exponential:

$$P(t) = e^{Qt}$$

This is defined as:

$$e^{Qt} = \sum_{k=0}^{\infty} \frac{(Qt)^k}{k!}$$

The state probability vector evolves as:

$$\mathbf{p}(t + \Delta t) \approx \mathbf{p}(t)\, e^{Q(x)\Delta t}$$

Eigenvalue Analysis

Eigenvalues determine the dynamics of the system.

	Eigenvalues		
Age	Female	Male	Both
40	(-0.0018, -0.1228, 0)	(-0.0047, -0.1274, 0)	(-0.0032, -0.1251, 0)
60	(-0.0109, -0.1374, 0)	(-0.0236, -0.1583, 0)	(-0.0165, -0.1465, 0)
80	(-0.0659, -0.2312, 0)	(-0.1160, -0.3225, 0)	(-0.0826, -0.2613, 0)

Interpretation

- Negative eigenvalues ensure stability
- Zero eigenvalue corresponds to the absorbing death state
- Increasing magnitude with age implies faster transitions

Probability Conservation

The model preserves total probability:

$$p_H(t) + p_D(t) + p_{\text{Dead}}(t) = 1$$

Numerical error is negligible ($< 10^{-15}$), confirming correctness.

Expected Present Value (EPV)

The model computes EPV discretely:

$$\text{EPV} = \sum_t B \cdot \Delta p_{\text{Dead}}(t) \cdot e^{-rt}$$

where:

- $B = 100{,}000$
- $r = 3\%$
- $\Delta p_{\text{Dead}}(t)$ is the incremental probability of death

Results

Population	EPV ($)
Female	31,034.65
Male	41,830.42
Both	36,288.98

Interpretation

Higher mortality leads to earlier payouts and thus higher EPV, explaining why male EPV is largest.

Sensitivity Analysis

λ_{12}	Female	Male	Both
0.01	30,178	41,071	35,438
0.02	31,035	41,830	36,289
0.05	32,925	43,544	38,180
0.10	34,781	45,288	40,062

Interpretation

- EPV increases with disability incidence
- More transitions into risk states increase expected payouts

Interpretation of Graphical Results

The simulated probability curves reflect the structure of the generator matrix.

Key Observations

- Healthy probability declines over time
- Death probability increases monotonically
- Disability exhibits a hump-shaped pattern

Linear Algebra Interpretation

The system evolves via:

$$\mathbf{p}(t) = \mathbf{p}(0)e^{Qt}$$

Eigenvalues determine the speed of convergence:

- Larger negative eigenvalues $\Rightarrow$ faster transitions
- Male population shows faster decay due to higher mortality

Disability Dynamics

The hump shape arises from competing flows:

- Inflow: λ_{12}
- Outflow: $\lambda_{21} + \mu_d(x)$

Code Walkthrough

This section explains the Python implementation in detail. The script is organized into eight modules, each corresponding to a stage of the modeling pipeline. All code is in a single file: `ctmc_actuarial_model.py`.

Dependencies

The script uses four external libraries:

```
import numpy as np
import matplotlib.pyplot as plt
from scipy.linalg import expm, eigvals
from sklearn.linear_model import LinearRegression
```

- `numpy`: numerical arrays and linear algebra primitives.
- `matplotlib`: plotting (Gompertz fit, state evolution, sensitivity).
- `scipy.linalg.expm`: computes the matrix exponential $e^{Q\Delta t}$. This is the core Linear Algebra operation in the project. Internally, `expm` uses a Padé approximation with scaling and squaring, which is numerically stable for matrices of this size.
- `scipy.linalg.eigvals`: computes eigenvalues of $Q(x)$ for the stability analysis.
- `sklearn.linear_model.LinearRegression`: performs the ordinary least squares (OLS) regression used to fit the Gompertz parameters.

To install any missing packages, run:

```
pip install numpy matplotlib scipy scikit-learn pandas
```

Module 1: Data Loading (`load_mortality_data`)

```
def load_mortality_data(filepath="mortality_data.csv",
                        age_min=30, age_max=90, year=2023):
    df = pd.read_csv(filepath)
    df = df[df["Year"] == year]
    ages = df["Age (x)"].values.astype(float)
    mx = df["Central death rate m(x,n)"].values.astype(float)
    mask = (ages >= age_min) & (ages <= age_max)
    ages = ages[mask]
    mx = mx[mask]
    Ex = np.ones_like(mx)
    Dx = mx
    return ages, Dx, Ex
```

What it does:

1. Reads a CSV file containing UN World Population Prospects life table data for Vietnam (2023 revision).
2. Filters to a single calendar year (default: 2023) and an age range of 30–90. Ages below 30 are excluded because the Gompertz law does not fit well at young ages (where accident and violence mortality dominate). Ages above 90 are excluded due to data sparsity.
3. Extracts the central death rate m_x, which is the observed force of mortality. Since m_x is already a rate (deaths per person-year), we set `Dx = mx` and `Ex = 1` so that the downstream code computes $D_x/E_x = m_x$ unchanged.
4. Returns three arrays: `ages`, `Dx`, `Ex`.

Data format: The CSV has columns including `Year`, `Age (x)`, and `Central death rate m(x,n)`. Since the UN data is in abridged 5-year age groups (30, 35, 40, . . . , 90), this produces 13 data points per run.

Switching datasets: To run for males or females, change the `filepath` argument to `mortality_data_male.cs` or `mortality_data_female.csv`.

Module 2: Gompertz Calibration (`fit_gompertz`)

```
def fit_gompertz(ages, Dx, Ex):
    lambda_hat = np.clip(Dx / Ex, 1e-10, None)
    X = ages.reshape(-1, 1)
    y = np.log(lambda_hat)
    reg = LinearRegression().fit(X, y)
    B = reg.coef_[0]
    A = np.exp(reg.intercept_)
    return A, B, lambda_hat
```

What it does:

1. Computes crude death rates: $\hat{\lambda}_x = D_x/E_x$. The `np.clip` call ensures no zero or negative values before taking logarithms.

2. Takes the natural log: $\log \hat{\lambda}_x$. Under the Gompertz model $\mu(x) = Ae^{Bx}$, this becomes:

$$\log \mu(x) = \underbrace{\log A}_{\text{intercept}} + \underbrace{B}_{\text{slope}} \cdot x$$

   which is a linear function of age.

3. Fits a linear regression of $\log \hat{\lambda}_x$ on x (age) using ordinary least squares. The slope gives B and the intercept gives $\log A$.

4. Recovers $A = e^{\text{intercept}}$.

Interpretation of parameters:

- A is the baseline mortality at age 0 (extrapolated). A larger A means higher mortality at all ages.

- B is the rate of exponential increase. Mortality doubles every $\ln 2/B$ years.

Note: This is a least-squares fit on log-transformed data, not a strict maximum likelihood estimator. For a course project this is standard and appropriate.

Module 3: Generator Matrix (`build_generator`)

```
def build_generator(age, A, B, lambda_HD, lambda_DH, DD_factor):
    mu_H = gompertz_hazard(age, A, B)
    mu_D = DD_factor * mu_H
    Q = np.array([
        [-(lambda_HD + mu_H),   lambda_HD,            mu_H  ],
        [  lambda_DH,          -(lambda_DH + mu_D),   mu_D  ],
        [  0,                    0,                    0     ]
    ])
    return Q
```

What it does:

Constructs the 3×3 generator matrix $Q(x)$ at a given age. This is the central object of the model. The function takes six arguments:

- `age`: the current age x, which determines the mortality rate via the Gompertz formula.
- `A, B`: the calibrated Gompertz parameters.
- `lambda_HD` ($\lambda_{12} = 0.02$): the rate of transitioning from Healthy to Disabled, per year.
- `lambda_DH` ($\lambda_{21} = 0.10$): the rate of recovery from Disabled to Healthy, per year.
- `DD_factor` ($= 2.0$): the disabled mortality multiplier. Disabled individuals face $2\times$ the mortality of healthy individuals at the same age.

Matrix structure:

$$Q(x) = \begin{bmatrix} -(\lambda_{12} + \mu_H) & \lambda_{12} & \mu_H \\ \lambda_{21} & -(\lambda_{21} + \mu_D) & \mu_D \\ 0 & 0 & 0 \end{bmatrix}$$

Key properties enforced by construction:

- Each row sums to zero (probability conservation).
- Off-diagonal entries are non-negative (valid transition rates).
- Row 3 is all zeros (Dead is an absorbing state: once entered, no transitions out).

Because $\mu_H = Ae^{Bx}$ depends on age, $Q(x)$ is different at every age. This is what makes the model "age-dependent" — the generator matrix changes as the individual ages.

Why not pass these as globals? In an earlier version of the code, `lambda_HD` and the other disability parameters were global variables. This caused a bug: the sensitivity analysis modified the global, and if the code ran in a different order, subsequent computations silently used the wrong value. The fix is to pass all parameters explicitly via a dictionary:

```
params = {
    "lambda_HD": 0.02,
    "lambda_DH": 0.10,
    "DD_factor": 2.0,
}
```

This dictionary is unpacked into function arguments using `**params`, e.g., `build_generator(age, A, B, **params)`.

Module 4: Eigenvalue Analysis (`analyze_generator`)

```
def analyze_generator(age, A, B, params):
    Q = build_generator(age, A, B, **params)
    evals = eigvals(Q)
    ...
    return Q, evals
```

What it does:

1. Builds $Q(x)$ at the given age.

2. Computes the eigenvalues of Q using `scipy.linalg.eigvals`.

3. Prints the matrix, eigenvalues, and two diagnostic checks:

   - Whether all non-zero eigenvalues have negative real parts (required for stability — ensures $e^{Qt} \to 0$ as $t \to \infty$ for transient states).
   - Whether a zero eigenvalue exists (confirms the presence of an absorbing state).

Linear Algebra connection: If Q is diagonalizable as $Q = PDP^{-1}$ where $D = \text{diag}(\lambda_1, \lambda_2, 0)$, then:

$$e^{Qt} = P \begin{bmatrix} e^{\lambda_1 t} & 0 & 0 \\ 0 & e^{\lambda_2 t} & 0 \\ 0 & 0 & 1 \end{bmatrix} P^{-1}$$

Since $\lambda_1, \lambda_2 < 0$, the terms $e^{\lambda_i t} \to 0$ as $t \to \infty$, meaning the system eventually converges to the absorbing state (death). The magnitude of the eigenvalues controls how fast this convergence occurs.

Module 5: State Probability Evolution (`evolve_probabilities`)

```
def evolve_probabilities(age_start, A, B, params, T=50, dt=0.1):
    n_steps = int(T / dt)
    times = np.arange(n_steps) * dt
    trajectory = np.zeros((n_steps, 3))
    state = np.array([1.0, 0.0, 0.0])

    for i, t in enumerate(times):
        trajectory[i] = state
        age = age_start + t
        Q = build_generator(age, A, B, **params)
        P_mat = expm(Q * dt)
        state = state @ P_mat

    return times, trajectory
```

What it does:
Simulates the probability distribution over the three states (Healthy, Disabled, Dead) for 50 years starting from age 40, using time steps of $\Delta t = 0.1$ years (i.e., 500 steps total).
Algorithm (step by step):

1. Initialize the state vector $\boldsymbol{\pi}(0) = [1, 0, 0]$, meaning the individual starts with 100% probability of being Healthy.

2. At each time step t:

   (a) Compute the current age: $x = 40 + t$.

   (b) Build the generator matrix $Q(x)$ at this age.

   (c) Compute the transition matrix over one time step: $P(\Delta t) = e^{Q(x) \cdot \Delta t}$. This is a 3×3 matrix where entry P_{ij} is the probability of transitioning from state i to state j in Δt years.

(d) Update the state vector: $\boldsymbol{\pi}(t + \Delta t) = \boldsymbol{\pi}(t) \cdot P(\Delta t)$. This is a row-vector–matrix multiplication.

3. Record $\boldsymbol{\pi}(t)$ at each step.

Why rebuild Q at every step? Because mortality $\mu(x) = Ae^{Bx}$ changes with age. At age 40, μ is small, so the transition matrix $P(\Delta t)$ is close to the identity (not much happens in 0.1 years). At age 80, μ is large, so $P(\Delta t)$ has substantial off-diagonal entries (significant probability of transitioning in 0.1 years).

Key operation: The line `state = state @ P_mat` is a matrix–vector product. The `@` operator in Python performs matrix multiplication. Since `state` is a row vector of shape (1×3) and `P_mat` is (3×3), the result is a new (1×3) row vector.

Module 6: Expected Present Value (`compute_EPV`)

```
def compute_EPV(age_start, A, B, params, r=0.03,
                benefit=100_000, T=50, dt=0.1):
    state = np.array([1.0, 0.0, 0.0])
    EPV = 0.0
    for t in np.arange(0, T, dt):
        age = age_start + t
        Q = build_generator(age, A, B, **params)
        P_mat = expm(Q * dt)
        next_state = state @ P_mat
        death_increment = next_state[2] - state[2]
        discount = np.exp(-r * t)
        EPV += benefit * death_increment * discount
        state = next_state
    return EPV
```

What it does:

Computes the expected present value (EPV) of a \$100,000 death benefit for an individual starting at age 40, discounted at $r = 3\%$ per year.

Formula (discrete approximation):

$$\text{EPV} = \sum_{k=0}^{N-1} B \cdot \Delta p_{\text{Dead}}(t_k) \cdot e^{-rt_k}$$

where:

- $B = 100{,}000$ is the death benefit,
- $\Delta p_{\text{Dead}}(t_k) = p_{\text{Dead}}(t_k + \Delta t) - p_{\text{Dead}}(t_k)$ is the incremental probability of dying during the interval $[t_k, t_k + \Delta t]$,
- e^{-rt_k} is the discount factor (a dollar received in the future is worth less today).

Step by step:

1. The loop is identical to Module 5: build $Q(x)$, compute $P(\Delta t) = e^{Q\Delta t}$, update the state vector.

2. At each step, compute how much the death probability increased: `next_state[2] - state[2]`. Index `[2]` is the Dead state (Python uses 0-based indexing).

3. Multiply by the benefit amount and the discount factor, and accumulate into `EPV`.

Interpretation: The EPV answers the question: "How much should an insurer set aside today to cover a $100,000 death benefit for a 40-year-old?" A higher EPV means the insurer expects to pay sooner (less discounting), which occurs when mortality is higher.

Module 7: Plotting Functions

Three plotting functions produce the project's visual outputs:

- `plot_gompertz_fit`: Two-panel figure showing observed vs. fitted mortality rates on both natural and log scales. The log-scale plot should appear roughly linear if the Gompertz model is appropriate.

- `plot_state_evolution`: Three curves (Healthy, Disabled, Dead) over time, with a secondary x-axis showing attained age. Uses color-coded lines: green for Healthy, orange for Disabled, red for Dead.

- `plot_sensitivity`: Plots EPV as a function of the disability inception rate λ_{12}, varying it from 0.005 to 0.10. This uses:

  ```
  test_params = {**base_params, "lambda_HD": lam}
  ```

  which creates a temporary copy of the parameters dictionary with only λ_{12} changed, leaving all other parameters untouched.

All plots are saved as PNG files at 150 DPI using `plt.savefig` and also displayed interactively via `plt.show()`.

Module 8: Main Function

```
def main():
    ages, Dx, Ex = load_mortality_data()
    A, B, lambda_hat = fit_gompertz(ages, Dx, Ex)
    ...
```

The `main()` function orchestrates the entire pipeline:

1. **Load data** from the CSV file.

2. **Fit Gompertz** parameters A and B via regression.

3. **Plot the Gompertz fit** (natural and log scale).

4. **Define disability parameters** in a dictionary:

   ```
   params = {
       "lambda_HD": 0.02,
       "lambda_DH": 0.10,
       "DD_factor": 2.0,
   }
   ```

5. **Eigenvalue analysis** at ages 40, 60, and 80.
6. **State evolution** from age 40 to 90.
7. **Probability conservation check**: verifies that $p_H(t) + p_D(t) + p_{\text{Dead}}(t) = 1$ at every time step. Deviations of 10^{-15} or smaller confirm numerical correctness.
8. **EPV computation** for the base case.
9. **Sensitivity analysis**: EPV at four disability rates (0.01, 0.02, 0.05, 0.10), plus a continuous plot over 20 values.

The guard `if __name__ == "__main__": main()` ensures that the script runs only when executed directly (not when imported as a module).

How to Run

1. Place `ctmc_actuarial_model.py` and the CSV data files in the same directory, or update the `filepath` argument in `load_mortality_data`.
2. Open a terminal and run:

   ```
   python ctmc_actuarial_model.py
   ```

3. The script will print numerical results to the console and display three plot windows sequentially. **Close each plot window** to proceed to the next stage.
4. Output files (`gompertz_fit.png`, `state_evolution.png`, `sensitivity.png`) are saved in the working directory.
5. To switch between datasets, change the file path to `mortality_data_male.csv` or `mortality_data_female.cs` and re-run.

Summary of Data Flow

The following diagram summarizes how data flows through the code:

$$\underbrace{\text{CSV}}_{\text{Module 1}} \xrightarrow{m_x} \underbrace{\text{Gompertz fit}}_{\text{Module 2}} \xrightarrow{A,\,B} \underbrace{Q(x)}_{\text{Module 3}} \xrightarrow{e^{Q\Delta t}} \underbrace{\boldsymbol{\pi}(t)}_{\text{Module 5}} \xrightarrow{\Delta p_{\text{Dead}}} \underbrace{\text{EPV}}_{\text{Module 6}}$$

Module 4 (eigenvalue analysis) branches off from Module 3 as a diagnostic tool, and Module 7 (plotting) visualizes the outputs of Modules 2, 5, and 6.

Conclusion

This project demonstrates how Linear Algebra provides the foundation for actuarial modeling:

- Matrix exponentials compute transition probabilities
- Eigenvalues explain system dynamics
- Real data calibration enhances realism

The integration of data, stochastic modeling, and Linear Algebra enables a rigorous valuation framework for insurance liabilities.